

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО  
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ  
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.  
Г. ШУХОВА» (БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и  
автоматизированных систем

**Лабораторная работа №2**  
по дисциплине: Основы ИИ  
тема: «Алгоритм теории адаптивного резонанса»

Выполнил: ст. группы ПВ-223  
Дмитриев Андрей Александрович

Проверил:  
Твердохлеб Виталий Викторович

Белгород 2025 г.

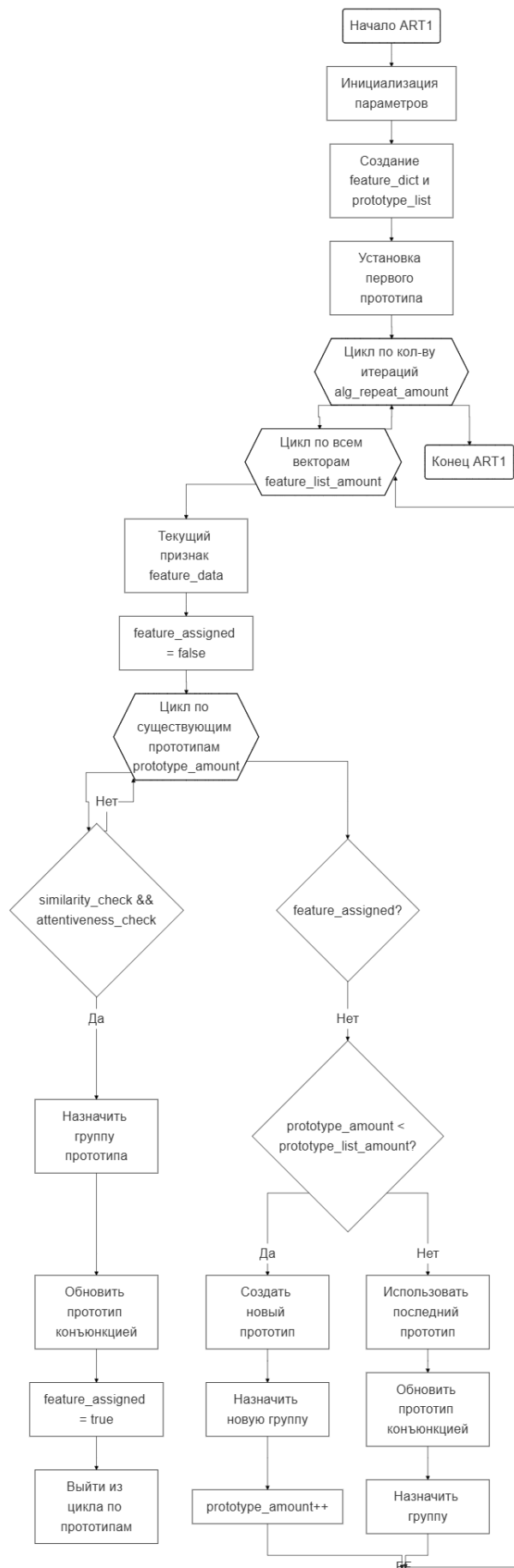
**Цель работы:** Изучение методики описания и технологии разработки алгоритма ART1 (Adaptive Resonance Theory) на примере решения задачи классификации.

**Содержание отчёта:**

1. Наименование и цель работы.
  2. Блок-схема алгоритма решения задачи N-ферзей с использованием метода обратного восстановления.
  3. Исходный код модулей программного обеспечения.
  4. Результаты исследования влияния параметров алгоритма на его эффективность, оформленные в виде таблицы.
1. Выводы по работе.

### Ход работы:

### Блок схема:



## Исходный код:

```
def similarity_check(prototype, feature, beta, list_size):
    """Проверка сходства между прототипом и признаком"""
    conjunction_value = get_list_conjunction_value(prototype, feature)
    prototype_value = sum(prototype)
    feature_value = sum(feature)

    return (conjunction_value / (beta + prototype_value)) > (feature_value /
(beta + list_size))

def attentiveness_check(prototype, feature, ro):
    """Проверка внимательности"""
    conjunction_value = get_list_conjunction_value(prototype, feature)
    feature_value = sum(feature)

    return (conjunction_value / feature_value) < ro or conjunction_value ==
feature_value

def get_list_conjunction_value(list1, list2):
    """Получить значение конъюнкции двух списков"""
    return sum(min(a, b) for a, b in zip(list1, list2))

def save_list_conjunction(prototype, feature):
    """Сохранить конъюнкцию списков (обновить прототип)"""
    for i in range(len(prototype)):
        prototype[i] = min(prototype[i], feature[i])

def feature_assign(feature_dict, prototype_list, prototype_amount, beta, ro,
list_size):
    """Назначение признаков прототипам"""
    feature_list_amount = len(feature_dict)
    prototype_list_amount = len(prototype_list)

    for f_list_id in range(feature_list_amount):
        feature_assigned = False
        feature_data = feature_dict[f_list_id]['data']

        for p_list_id in range(prototype_list_amount):
            if (similarity_check(prototype_list[p_list_id], feature_data,
beta, list_size) and
feature_data, ro)):
                attentiveness_check(prototype_list[p_list_id],
feature_data, ro)):
                    feature_dict[f_list_id]['group'] = p_list_id
                    save_list_conjunction(prototype_list[p_list_id],
feature_data)
                    feature_assigned = True
                    break

        if not feature_assigned:
            if prototype_amount < prototype_list_amount:
                prototype_list[prototype_amount] = feature_data[:]
                feature_dict[f_list_id]['group'] = prototype_amount
                prototype_amount += 1
            else:
                save_list_conjunction(prototype_list[prototype_amount - 1],
feature_data)
                feature_dict[f_list_id]['group'] = prototype_amount - 1

    return prototype_amount

def ART1(feature_list_amount, prototype_list_amount, list_size, data, beta,
```

```

ro, alg_repeat_amount):
    """Основная функция алгоритма ART1"""
    if alg_repeat_amount < 1:
        raise ValueError("Cannot repeat algorithm less than 1 time")

    # Инициализация словаря признаков
    feature_dict = {}
    for i in range(feature_list_amount):
        feature_dict[i] = {
            'data': data[i][:], # Копируем данные
            'group': -1 # Инициализируем группу как -1
        }

    # Инициализация списка прототипов
    prototype_list = [[0] * list_size for _ in range(prototype_list_amount)]
    prototype_amount = 0

    # Инициализация: первый прототип = первый признак
    prototype_list[0] = feature_dict[0]['data'][:]
    feature_dict[0]['group'] = 0
    prototype_amount = 1

    # Выполнение алгоритма
    for i in range(alg_repeat_amount):
        prototype_amount = feature_assign(
            feature_dict, prototype_list, prototype_amount, beta, ro,
list_size
        )

    return feature_dict, prototype_list[:prototype_amount]

def use_sample_data():
    """Использование примерных данных"""
    prototype_list_amount = 3
    beta = 0.5
    ro = 0.8
    alg_repeat_amount = 2

    # Пример данных (бинарные векторы)
    data = [
        [0, 1, 0, 0, 0, 0, 0, 1, 0],
        [0, 0, 0, 1, 0, 0, 1, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 1, 0],
        [1, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 1, 0, 0, 1],
        [1, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0],
        [0, 0, 1, 0, 0, 1, 0, 0, 1],
    ]

    feature_list_amount = len(data)
    list_size = len(data[0])

    return feature_list_amount, prototype_list_amount, list_size, data,
beta, ro, alg_repeat_amount

USE_COMMAND_INPUT = False
SHOW_CLUSTERING_RESULT = True
SHOW_PROTOTYPES_RESULT = True

if __name__ == "__main__":
    if USE_COMMAND_INPUT:
        feature_list_amount = int(input("Введите количество признаков: "))
        list_size = int(input("Введите размер вектора признаков: "))
        prototype_list_amount = int(input("Введите максимальное количество

```

```

прототипов: "))
    beta = float(input("Введите параметр beta: "))
    ro = float(input("Введите параметр ro: "))
    alg_repeat_amount = int(input("Введите количество повторений
алгоритма: "))

    print(f"\nВведите {feature_list_amount} векторов (по {list_size}
чисел 0 или 1 в каждом):")
    data = []
    for i in range(feature_list_amount):
        print(f"Вектор {i + 1}: ", end="")
        vector_input = input()
        vector = list(map(int, vector_input.split()))
        data.append(vector)

    else:
        (feature_list_amount,
         prototype_list_amount,
         list_size,
         data,
         beta,
         ro,
         alg_repeat_amount) = use_sample_data()

    # Запуск алгоритма
    feature_dict, prototypes = ART1(
        feature_list_amount, prototype_list_amount, list_size,
        data, beta, ro, alg_repeat_amount
    )

    # Вывод результатов
    if SHOW_CLUSTERING_RESULT:
        print("\nРезультаты кластеризации:")
        for i in range(feature_list_amount):
            print(f"Признак {i}: {feature_dict[i]['data']} -> Группа
{feature_dict[i]['group']}")

    if SHOW_PROTOTYPES_RESULT:
        print("\nФинальные прототипы:")
        for i in range(len(prototypes)):
            print(f"Прототип {i}: {prototypes[i]}")

```

## Пример работы программы:

```

Результаты кластеризации:
Признак 0: [0, 1, 0, 0, 0, 0, 0, 1, 0] -> Группа 0
Признак 1: [0, 0, 0, 1, 0, 0, 1, 0, 0] -> Группа 1
Признак 2: [0, 0, 0, 0, 1, 0, 0, 1, 0] -> Группа 0
Признак 3: [1, 0, 0, 1, 0, 0, 0, 0, 0] -> Группа 1
Признак 4: [0, 0, 0, 0, 0, 1, 0, 0, 1] -> Группа 2
Признак 5: [1, 0, 0, 1, 0, 0, 0, 0, 0] -> Группа 1
Признак 6: [0, 0, 0, 0, 1, 0, 0, 0, 0] -> Группа 2
Признак 7: [0, 0, 1, 0, 0, 1, 0, 0, 1] -> Группа 2

Финальные прототипы:
Прототип 0: [0, 0, 0, 0, 0, 0, 0, 1, 0]
Прототип 1: [0, 0, 0, 1, 0, 0, 0, 0, 0]
Прототип 2: [0, 0, 0, 0, 0, 0, 0, 0, 0]

```

**Эксперименты:**

| beta | ro  | feature amount | prototype amount |
|------|-----|----------------|------------------|
| 1    | 0.7 | 10             | 8                |
| 1    | 0.7 | 10             | 5                |
| 1    | 0.7 | 10             | 3                |
| 1    | 0.3 | 10             | 7                |
| 1    | 0.3 | 10             | 3                |
| 5    | 0.7 | 10             | 8                |
| 5    | 0.7 | 10             | 5                |
| 5    | 0.7 | 10             | 3                |
| 5    | 0.3 | 10             | 7                |
| 5    | 0.3 | 10             | 3                |

**Вывод:** В ходе лабораторной работы алгоритм кластеризации ART1 отличается простотой реализации и высокой гибкостью. Высокая гибкость по большей части обуславливается 3 параметрами:  $\beta$ ,  $\gamma$  и количеством векторов-прототипов. Параметр  $\beta$  отвечает за сложность прохождения проверки на схожесть, чем параметр больше, тем проверка сложнее. Параметр  $\gamma$  отвечает за сложность проверки на внимательность, чем меньше  $\gamma$ , тем сложнее проверка.